



Getting Started with Senders and Receivers

James Pascoe

GETTING STARTED WITH SENDERS AND RECEIVERS

(A 20-MINUTE LEARNING GUIDE AND PRIMER)

Jim (James) Pascoe

<http://www.james-pascoe.com>

james@james-pascoe.com

<http://jamespascoe.github.io/accu2025-senders-and-receivers>

<https://github.com/jamespascoe/accu2025-example-code>

ACCU Bristol and Bath Meetup Coordinator

HOW FAMILIAR ARE YOU WITH SENDERS AND RECEIVERS?

1. I am a sender/receiver expert
2. I have a strong grasp but need to fill in some details
3. I have some understanding - its a work-in-progress
4. I am just starting to learn about them

SENDERS AND RECEIVERS

- Standardised asynchronous programming
 - **Composability** between libraries
 - network → thread pool → embedded io
 - no 'glue' code
 - complement coroutines (not either/or)

ASIO TIMER: SENDER

```
1 struct timer_sender {
2     using sender_concept = stdexec::sender_t;
3
4     using completion_signatures =
5         stdexec::completion_signatures<
6             stdexec::set_value_t(int),
7             stdexec::set_error_t(int)>
8
9     auto connect(stdexec::receiver auto rcvr) {
10         asio::steady_timer timer(io, asio::chrono::seconds(n_secs));
11
12         std::printf("Created %d second timer\n", n_secs);
13
14         return timer_op {
15             io, n_secs, std::move(timer), std::move(rcvr)
16         };
17     };
18
19     asio::io_context& io;
20     int n_secs;
21 };
```

ASIO TIMER: RECEIVER

```
1  template <class Receiver>
2  struct timer_op {
3      asio::io_context& io;
4      int n_secs;
5      asio::steady_timer timer;
6      Receiver rcvr;
7
8      static void callback(const boost::system::error_code& error,
9                          int n_secs, Receiver& rcvr)
10     {
11         std::printf("Finished %d second timer\n", n_secs);
12
13         if (error != boost::system::errc::success)
14             stdexec::set_error(std::move(rcvr), error);
15         else
16             stdexec::set_value(std::move(rcvr), 0);
17     }
18
19     void start() noexcept
20     {
21         timer.async_wait(
```

ASIO TIMER: MAIN CODE

```
1 timer_sender async_timer(asio::io_context& io, int n_secs) {  
2     return {io, n_secs};  
3 }  
4  
5 int main() {  
6     asio::io_context io;  
7  
8     auto task_timer = async_timer(io, 5);  
9     auto print_ret_val = stdexec::then(task_timer, [](int n)  
10    {  
11        std::printf("Return value: %d\n", n);  
12    });  
13  
14    stdexec::sync_wait(print_ret_val);  
15 }
```

ASIO TIMER: CMAKE

```
1 cmake_minimum_required(VERSION 3.25)
2
3 set (CMAKE_CXX_STANDARD 20)
4
5 project(ACCU2025)
6
7 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
8
9 # Use the CMake Package Manager
10 include(CPM)
11
12 # Download stdexec
13 CPMAddPackage(
14     NAME stdexec
15     GITHUB_REPOSITORY NVIDIA/stdexec
16     GIT_TAG main
17 )
18
19 set (STDEXEC_BUILD_TESTS OFF)
20
21 set(THREADS_PREFER_PTHREAD_FLAG ON)
```


ADAPTORS

- Algorithms for common asynchronous patterns
 - **then**: input sender → function
 - **continues_on**: change execution resource
 - **bulk**: runs a task for every index in a space
- Most adaptors are pipeable: |
- **cppreference** and **std::exec** have good examples

COROUTINES

- Senders and Receivers work well with coroutines
- All awaitables are senders
- `task<>` type in `std::exec` knows how to await senders
- You can `co_await` without having to do extra work
- More information in key references

KEY REFERENCES

- Eric Niebler: [What are senders good for, Anyway?](#)
- Lucian Radu Teodorescu: Overload [Dec '24](#) & [Feb '25](#)
- Goran Arandjelović: [CppNow 2024](#) talk
- Dietmar Kühl: [ACCU 2024](#) (networking) talk
- Ben Deane: [CppNow 2024](#) (embedded) talk
- Nvidia [std::exec](#) resources

ADVICE FOR LEARNING

- Start with small examples e.g. Eric's
- Use [P2300](#) as a reference ... but later
- Persevere with the key references
- Make use of the example code
- Thoroughly understand concepts

QUESTIONS?

<http://www.james-pascoe.com>

james@james-pascoe.com

<http://jamespascoe.github.io/accu2025-senders-and-receivers>

<https://github.com/jamespascoe/accu2025-example-code>

ACCU Bristol and Bath Meetup Coordinator